


# Documento Técnico Breve de ClienTrack: Arquitectura, Datos y Validaciones

Yiseth Karina Gomez Hernandez

Universidad EAN

Desarrollo Web

Diego Fernando Chinchilla Beltrán

3 de noviembre de 2026

## ClienTrack: arquitectura, datos y validaciones

Este documento resume las decisiones técnicas de ClienTrack, una aplicación Android en Java para gestionar clientes y sus casos.

## Patrón arquitectónico

Se eligió el patrón Modelo-Vista-Controlador (MVC). En Android, las Activities cumplen el papel de controlador: reciben las acciones del usuario, validan los datos y deciden qué mostrar. Las vistas son los archivos XML de res/layout, que solo describen la interfaz, y el modelo está formado por las clases Usuario, Cliente y Caso junto con DatabaseHelper, la única clase que ejecuta consultas SQL (ver Tabla 1).

Se prefirió MVC sobre MVVM porque es el patrón trabajado en la asignatura y porque el proyecto usa SQLiteOpenHelper sin componentes de Jetpack como ViewModel o LiveData, que son los que le dan sentido a MVVM. Aun así, la separación se cumple: ninguna pantalla escribe SQL, de modo que un cambio en la base de datos solo afecta a DatabaseHelper, y las reglas de validación están en una clase aparte (Validador) que se puede probar sin emulador.

**Tabla 1**

*Capas del patrón MVC en ClienTrack*

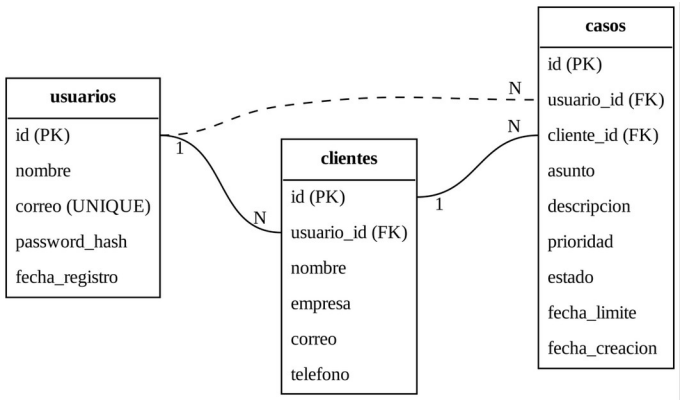
| Capa | Elementos | Responsabilidad |
|---|---|---|
| Modelo | Usuario, Cliente, Caso y DatabaseHelper | Representar los datos y ejecutar todas las consultas a SQLite |
| Vista | Archivos XML de res/layout | Definir la apariencia de cada pantalla |
| Controlador | Activities del paquete ui | Recibir eventos, validar, pedir datos al modelo y actualizar la vista |

## Esquema de datos y persistencia

Los datos se guardan en una base SQLite local llamada clientrack.db, creada con SQLiteOpenHelper. Como muestra la Figura 1, un usuario tiene muchos clientes y cada cliente puede tener muchos casos. La tabla casos también guarda usuario_id para filtrar todas las consultas por el usuario que inició sesión.

**Figura 1**

*Modelo entidad-relación de la base de datos*




*Nota.* PK = llave primaria; FK = llave foránea.

Las llaves foráneas usan ON DELETE CASCADE, que se activa en onConfigure, para que al borrar un cliente no queden casos huérfanos. El correo del usuario es UNIQUE y las columnas prioridad y estado tienen restricciones CHECK con sus valores permitidos. Las fechas se guardan como texto en formato yyyy-MM-dd, que se puede ordenar y comparar directamente. La sesión se guarda en SharedPreferences solo con el id y el nombre del usuario, nunca con la contraseña.

## Validaciones principales

Los errores aparecen debajo del campo que los causa y cada acción exitosa se confirma en pantalla (ver Tabla 2).

**Tabla 2**

*Validaciones de los formularios*

| Formulario | Campo | Regla |
|---|---|---|
| Registro | Nombre | Obligatorio; solo letras y espacios, de 3 a 60 caracteres |
| Registro | Correo | Obligatorio, con formato válido y sin haber sido registrado antes |
| Registro | Contraseña | Mínimo 6 caracteres e igual a la confirmación; se guarda con SHA-256 |
| Inicio de sesión | Correo y contraseña | Obligatorios; la contraseña se compara con el hash guardado |
| Cliente | Nombre | Obligatorio; solo letras y espacios, de 3 a 60 caracteres |
| Cliente | Correo y teléfono | Opcionales; si se llenan, correo válido y teléfono de 7 a 10 dígitos |
| Caso | Cliente | Debe existir; si no hay clientes, la app pide crear uno primero |
| Caso | Asunto y fecha límite | Asunto de al menos 3 caracteres; fecha obligatoria y, si el caso es nuevo, no anterior a hoy |